Programación Orientada a Objetos
Boletín 2

El objetivo de este boletín es hacer uso de los distintos tipos de clases que ofrece Java para la manipulación de conjuntos de datos. Más concretamente, se trabajarán los siguientes aspectos:

- Uso de **`ArrayList`**, para almacenar conjuntos de datos sobre los que es necesario iterar.
- Uso de **`Set`** y **`HashSet`**, para almacenar conjuntos de datos que no se pueden repetir.
- Uso de **`HashMap`**, para almacenar conjuntos de datos a los que se accede habitualmente a través de una clave.
- Uso de **`for-each`** para recorrer los elementos de una **`Collection`**.
- Uso de **`iterator`** para recorrer los elementos de un **`HashMap`**.
- Uso de los métodos **`equals`** y **`hashCode`** para acceder a los elementos de los conjuntos de datos.

Para ello, en los ejercicios de este boletín se crearán las diferentes clases que representarán las características de un sensor; una estancia, en la que están instalados varios sensores; una planta, que contiene varias estancias; y un edificio, que está formado por varias plantas.

Ejercicio 1

1. Crear un proyecto denominado `B2E1` en el que se crearán tres clases: una clase `Sensor`, que representa las características principales de un sensor; una clase `Estancia`, que representa las características de la estancia del edificio en la que se ha instalado el sensor; y la clase `PrincipalB2E1` en la que se crearán instancias/objetos de las clases `Sensor` y `Estancia` para testear que los métodos y constructores están correctamente implementados.

2. Crear la clase que representa las características de un sensor.

Nombre de la clase: `Sensor`

Atributos:

- `id` (`String`): identificador del sensor.
- `tipo` (`String`): tipo de sensor.
- `estancia` (`Estancia`): nombre de la estancia en la que se encuentra instalado el sensor.
- `datos` (`ArrayList<Float>`): conjunto de datos que ha sido capturado por el sensor. El dato más reciente deberá estar en la última posición y el más antiguo en la primera.
- `rango` (`Float[]`): rango de valores que puede tomar el sensor.
- `precio` (`Float`): precio del sensor.

Constructores:

Se debe implementar el siguiente constructor, en el que se deben cumplir las restricciones de valores indicadas para los setters de los atributos:

- Constructor con los siguientes argumentos: `id` (`String`), `tipo` (`String`), `rango` (`float[]`) y `precio` (`float`).

Adicionalmente, se considerarán las siguientes restricciones:

- Se debe evitar que se pueda hacer aliasing sobre el atributo `rango`.

- El atributo `tipo` podrá tomar los siguientes valores: "temperatura", "sonido", "luz" y "humedad".

Getters/Setters:

Los atributos de la clase deben tener los correspondientes getters, salvo para el atributo `rango` que no debe tener getter.

El atributo `estancia` debe tener el setter implementado.

Métodos funcionales:

- `public float limiteInferior()`: devuelve el valor mínimo que puede tomar el sensor.
- `public float limiteSuperior()`: devuelve el valor máximo que puede tomar el sensor.
- `public void nuevoDato(float dato)`: introducir un nuevo `dato` capturado por el sensor.
- `public float media()`: devuelve la media de los valores del sensor.
- `public float maximo()`: devuelve el máximo de los valores del sensor.
- `public float minimo()`: devuelve el mínimo de los valores del sensor.
- `public boolean equals(Object object)`: donde se considera que dos sensores son iguales si sus identificadores son iguales.
- `public String toString()`: la representación en cadena de texto que debe devolver seguirá la siguiente forma:

```
{
 "id": "<valor>",
 "tipo": "<valor>",
 "estancia": "<nombre_estancia>",
 "rango": [ "<valor1>", "<valor2>" ],
 "valores": [ "<valor1>", "<valor2>",..., "<valor10>" ],
 "precio" : "<valor>",
 "mínimo" : "<valor>",
 "máximo" : "<valor>",
 "media" : "<valor>"
}
```

donde `<valor>` es el valor del atributo correspondiente (para el atributo `valores` solamente se mostrarán sus últimos 10 valores; aunque no hay límite en el número de datos almacenados). Además, si los atributos no tienen valores (es decir, son `null`), entonces no formarán parte de la representación en cadena de texto del objeto correspondiente.

3. Crear la clase que representa las características de una estancia.

Nombre de la clase: `Estancia`

Atributos:

- `nombre` (`String`): nombre de la estancia, que también es su identificador único.
- `planta (int)`: planta en la que se encuentra la estancia.
- `sensores` (`Set<Sensor>`): conjunto de sensores que están instalados en la estancia.
- `coste` (`float`): coste de la estancia, entendida como la suma del precio de todos los sensores de la estancia.

Constructores:

Se debe implementar el siguiente constructor, en el que se deben cumplir las restricciones de valores indicadas para los setters de los atributos:

- Constructor con los siguientes argumentos: `nombre` (`String`) y `planta` (`int`).

Getters/Setters:

Todos los atributos de la clase deben tener los correspondientes getters implementados. No se implementará ningún setter.

Métodos funcionales:

- `public boolean darAlta(Sensor sensor)`: dar de alta un `sensor`, teniendo en cuenta el criterio de igualdad definido en la clase `Sensor`.
- `public boolean darAlta(String id, String tipo, float[] rango, precio float)`: dar de alta un sensor a partir de su `id`, `tipo`, `rango` y `precio`, teniendo en cuenta el criterio de igualdad.
- `public boolean nuevoDato(String id, float dato)`: introducir un nuevo `dato` capturado por el sensor que tiene un determinado identificador `id`, devolviendo `true` si ha sido introducido correctamente o `false`, en caso contrario.
- `public Set<Sensor> sensoresTipo(String tipo)`: devuelve el conjunto de sensores de un determinado `tipo` que hay en la estancia.
- `public Sensor[] sensorMaxValor(String tipo)`: devuelve el sensor (o sensores) de un determinado `tipo` que ha capturado el valor máximo.
- `public Sensor[] sensorMinValor(String tipo)`: devuelve el sensor (o sensores) de un determinado `tipo` que ha capturado el valor mínimo.
- `public ArrayList<Float> valoresSuperiores(String id, float valor)`: devuelve el conjunto de valores de un sensor con identificador `id` que superan un determinado `valor`.
- `public boolean equals(Object object)`: donde se considera que dos estancias son iguales si sus nombres son iguales y si la planta en la que se encuentran es la misma.
- `public String toString()`: la representación en cadena de texto que debe seguirá la siguiente forma:

```
{
 "nombre": "<valor>",
 "planta": "<id>",
 "sensores": [ "<id1>", "<id2>",..., "<idK>" ],
 "coste" : "<valor>",
 "sensores_media": [ { "<id1>", "media1" },
                     { "<id2>", "media2" },
                     ...,
                     { "<idK>", "mediaK" } ],
}
```

donde `<valor>` es el valor del atributo correspondiente, `id` es el identificador de un sensor, y `mediaK` es el valor medio de los valores capturados por el sensor con `idK`. Además, si los atributos no tienen valores (es decir, son `null`), entonces no formarán parte de la representación en cadena de texto del objeto correspondiente.

4. Crear la clase principal del programa en la que se realizarán las siguientes operaciones:

Nombre de la clase: `PrincipalB2E1`

Operaciones a realizar en la función `main()`:

a. Crear los sensores `sensor1`, `sensor2`, `sensor3` y `sensor4` con los argumentos indicados en el fichero Excel (B2E1_instancias.xls).

b. Mostrar por pantalla los límites superiores e inferiores de los rangos de valores para todas las instancias creadas en (**a**).

c. Crear una estancia denominada `estancia1` cuyo `nombre` es "Aula01" y la `planta` es 2.

d. En la estancia `estancia1`, dar de alta a los sensores creados en (**a**), usando el método que tiene como argumento la instancia de la clase `Sensor` e indicando si el sensor se ha dado de alta correctamente o no.

e. Introducir los valores de los sensores creados en (**a**), que están en el fichero Excel (B2E1_instancias.xls), usando el método `nuevoDato` de la clase `Estancia` y mostrando por pantalla si el dato ha sido introducido correctamente o no.

f. Mostrar por pantalla la representación en texto de los sensores creados en (**a**).

g. Mostrar por pantalla la representación en texto de `estancia1`.

h. En la `estancia1`, dar de alta a los sensores `sensor5`, `sensor6` y `sensor7` con los argumentos especificados en el fichero Excel, indicando si el sensor se ha dado de alta correctamente o no. Se deberá utilizar el método de la clase `Estancia` cuyos argumentos son los valores de los atributos de los sensores.

i. Mostrar por pantalla los límites superiores e inferiores de los rangos de valores para todas las instancias creadas en (**h**).

j. Mostrar por pantalla la representación en texto de los sensores creados en (**h**).

k. Mostrar por pantalla la representación en texto de `estancia1`.

l. Introducir los valores de los sensores creados en (**h**) que están indicados en el fichero Excel, usando el método `nuevoDato` de la clase `Estancia` y mostrando por pantalla si cada dato ha sido introducido correctamente o no.

m. Mostrar por pantalla la representación en texto de `estancia1`.

n. Mostrar por pantalla las características de los sensores de "temperatura" que hay en `estancia1`.

o. Mostrar por pantalla las características de los sensores de "sonido" que hay en `estancia1`.

p. Mostrar por pantalla las características del sensor de "humedad" que ha capturado el valor máximo y el mínimo en la `estancia1`.

q. Para el sensor cuyo `id` es 301, mostrar por pantalla la lista de valores mayores de 33.

r. En `estancia1`, dar de alta al sensor `sensor8` con los argumentos especificados en el fichero Excel que se adjunta, indicando si el sensor se ha dado de alta correctamente o no. Se deberá utilizar el método de la clase `Estancia` cuyos argumentos son los valores de los atributos del sensor.

s. Mostrar por pantalla la representación en texto de `estancia1`.

Ejercicio 2

1. Crear un proyecto denominado `B2E2` en el que se hará uso de las mismas clases que las del `Ejercicio1`, incluyendo las modificaciones sobre esas clases que se indican a continuación.

2. Para la clase `Sensor`:

Atributos: son los mismos que los del `Ejercicio1`.

Constructores: son los mismos que los del `Ejercicio1`.

Getters/Setters: son los mismos que los del `Ejercicio1`.

Métodos funcionales: son los mismos que los del `Ejercicio1`, pero teniendo en cuenta las siguientes restricciones sobre los métodos indicados:

- `public float media()`: devuelve la media de los valores del sensor. Los datos se deberán recorrer con un `for-each`.
- `public float maximo()`: devuelve el máximo de los valores del sensor. Los datos se deberán recorrer con un `for-each`.
- `public float minimo()`: devuelve el mínimo de los valores del sensor. Los datos se deberán recorrer con un `for-each`.

3. Para la clase `Estancia`:

Atributos: son los mismos que en el `Ejercicio1`, pero teniendo en cuenta las siguientes modificaciones:

- `sensores` (`HashMap<String, Sensor>`): conjunto de sensores que están instalados en la estancia, donde la clave es el `id` del sensor.
- `coste` (`float`): coste de la estancia, entendida como la suma del precio de todos los sensores de la estancia.

Constructores: son los mismos que en el `Ejercicio1`, pero hay que adaptarlos al uso de un `HashMap` en vez de un `Set`.

Getters/Setters: son los mismos que en el `Ejercicio1`, pero hay que adaptarlos al uso de un `HashMap` en vez de un `Set`.

Métodos funcionales: son los mismos que en el `Ejercicio1`, pero hay que adaptarlos al uso de un `HashMap` en vez de un `Set`. Además, se introducen las siguientes modificaciones:

- `public Set<Sensor> sensoresTipo(String tipo)`: devuelve el conjunto de sensores de un determinado `tipo` que hay en la estancia. El conjunto de sensores se debe recorrer usando un `iterator` sobre las claves.
- `public Sensor[] sensorMaxValor(String tipo)`: devuelve el sensor (o sensores) de un determinado `tipo` que ha capturado el valor máximo. El conjunto de sensores se debe recorrer usando un `iterator` sobre los datos.
- `public Sensor[] sensorMinValor(String tipo)`: devuelve el sensor (o sensores) de un determinado `tipo` que ha capturado el valor mínimo. El conjunto de sensores se debe recorrer usando un `for-each` sobre las claves.
- `public ArrayList<Float> valoresSuperiores(String id, float valor)`: devuelve el conjunto de valores de un sensor con identificador `id` que superan un determinado `valor`. El conjunto de sensores se debe recorrer usando un `for-each` sobre los datos.

Y adicionalmente, se crearán los siguientes métodos:

- `public HashMap<String, ArrayList<Float>> valoresSensoresTipo(String tipo)`: devuelve el conjunto de valores de todos los sensores de un determinado `tipo`, donde la clave es el `id` del señor y los datos son los valores de dicho sensor. El conjunto de sensores se debe recorrer usando un `iterator` sobre las claves.
- `public ArrayList<Sensor> sensoresCosteMayor(float valor)`: devuelve el conjunto de sensores de que tienen un coste mayor que un determinado `valor`. El conjunto de sensores de `Estancia` se debe recorrer usando un `iterator` sobre los `Map.Entry<String, Sensor>`.

4. Crear la clase principal del programa en la que se realizarán las siguientes operaciones:

Nombre de la clase: `PrincipalB2E2`

Operaciones a realizar en la función `main()`: las mismas operaciones que las del `Ejercicio1`. Además, se realizarán las siguientes operaciones:

t. Mostrar por pantalla los valores de todos los sensores de "ruido" usando para ello el método `valoresSensoresTipo`.

u. Mostrar por pantalla los identificadores de los sensores que tienen un coste mayor de 40.

Ejercicio 3

1. Crear un proyecto denominado `B2E3` en el que se crearán tres clases: una clase `Planta`, que representan las características de una planta en la que se encuentran las estancias en las que están los sensores; una clase `Edificio`, que representa las características del edificio que tiene un conjunto de plantas; y la clase `PrincipalB2E3` en la que se crearán instancias/objetos de las clases `Planta` y `Edificio` para testear que los métodos y constructores están correctamente implementados. Por otra parte, también se cambiará la clase `Estancia` para que permita dar de baja sensores que se han estropeado y para modificar el tipo del atributo `planta` de dicha clase.

2. Modificar la clase `Estancia`:

Atributos: son los mismos que los del `Ejercicio2`, cambiando el tipo del atributo `planta` para que sea `Planta` en vez de `int`.

Constructores: es el mismo que el del `Ejercicio2`, cambiando el tipo argumento del constructor para que sea `planta (Planta)`.

Getters/Setters: son los mismos que los del `Ejercicio2`, cambiando el tipo que devuelve el getter de `planta`.

Métodos funcionales: son los mismos que los del `Ejercicio2`, añadiendo el siguiente método:

- `public boolean darBaja(String id)`: da de baja al sensor con identificador `id`, de modo que si se da de baja exitosamente, es decir, existe un sensor con ese `id`, se devuelve `true`, en caso contrario, se devolverá `false`.

3. Crear la clase que representa las características de una planta.

Nombre de la clase: `Planta`

Atributos:

- `numero` (`int`): número de planta.
- `tipo` (`String`): tipo de planta.
- `edificio` (`Edificio`): edificio en el que se encuentra la planta.
- `estancias` (`HashMap<String, Estancia>`): conjunto de estancias que forman parte de la planta.
- `presupuesto` (`float`): presupuesto destinado a la compra de los sensores y a su mantenimiento a lo largo del tiempo. Su valor es la suma del precio de todos los sensores de todas las estancias de la planta (es decir, la suma del coste de todas las estancias de la planta) más el 10% de esa suma. Hay que considerar que los sensores nuevos que se den de alta por la substitución de sensores que están defectuosos también contribuyen al presupuesto.
- `sensoresConProblemas` (`HashMap<String, Sensor>`): conjunto de sensores de la planta que han tenido problemas y que fueron substituidos por otros sensores, donde la clave indica el sensor que ha tenido problemas y el dato indica el sensor que lo ha substituido.

Constructores:

Se deben implementar los siguientes constructores, en los que se deben cumplir las restricciones de valores indicadas para los setters de los atributos:

- Constructor con los siguientes argumentos: `numero` (`int`), `tipo` (`String`), `edificio` (`Edificio`) y `estancias` (`HashMap<String, Estancia>`).
- Constructor con los siguientes argumentos: `numero` (`int`) y `tipo` (`String`).

Adicionalmente, se considerarán las siguientes restricciones:

- El atributo `tipo` podrá tomar los siguientes valores: "oficinas", "viviendas libres", "viviendas de protección oficial" y "administración".

Getters/Setters:

Los atributos de la clase deben tener los correspondientes getters.

Se deben implementar los setters de los atributos `estancias`, `edificio` y `tipo`, donde `tipo` debe cumplir con las restricciones indicadas anteriormente.

Métodos funcionales:

- `public boolean darAltaEstancia(ArrayList<Estancia> estancias)`: dar de alta un conjunto de estancias `estancias`. Se deberá comprobar si el número de planta coincide con el identificador la planta en la que se dará de alta. En caso de que no sea así, se devolverá `false`.
- `public boolean darAltaEstancia(Estancia estancia)`: dar de alta una `estancia`. Se deberá comprobar si el número de planta coincide con el identificador la planta en la que se dará de alta. En caso de que no sea así, se devolverá `false`.
- `public void darAltaSensor(String id, Sensor sensor)`: dar de alta un `sensor` en la estancia con identificador `id`. Hay que tener en cuenta que al dar de alta un sensor es necesario actualizar el presupuesto de la planta.
- `public void darAltaSensor(String id, ArrayList<Sensor> sensores)`: dar de alta un conjunto de `sensores` en la estancia con identificador `id`. Hay que tener en cuenta que al dar de alta un sensor es necesario actualizar el presupuesto.
- `public void darBajaSensor(String id, Sensor sensor)`: dar de baja a un sensor con identificador `id` y substituirlo por otro sensor `sensor`. Hay que tener en cuenta que es necesario actualizar tanto el conjunto de sensores con problemas como presupuesto de la planta.
- `public Set<Sensor> sensoresTipo(String tipo)`: devuelve el conjunto de sensores de un determinado `tipo` que hay en la planta. Todas las iteraciones sobre los datos deben hacerse usando un `iterator` sobre los valores.
- `public Set<String> tipoSensoresMasDefectuosos()`: devuelve los tipos de sensores que han tenido que ser substituidos en algún momento por algún defecto. Por ejemplo, si en la planta dos sensores del tipo "humedad", un sensor del tipo "sonido" ha sido substituido y no ha sido substituido ningún sensor del tipo "temperatura", se devolverá un `Set` que contendrá "sonido" y "humedad". Todas las iteraciones sobre los datos deben hacerse usando un `iterator` sobre las claves.
- `public boolean equals(Object object)`: donde se considera que dos plantas son iguales si sus números son iguales y si el edificio en el que se encuentra es el mismo.
- `public String toString()`: la representación en cadena de texto que debe devolver seguirá la siguiente forma:

```
{
 "numero": "<valor>",
 "tipo": "<valor>",
 "edificio": "<id>",
 "estancias": [ "<nombre_1>", "<nombre_2>",..., "<nombre_n>" ],
 "presupuesto" : "<valor>",
 "sensoresConProblemas" : "<id_1>", "<id_2>",..., "<id_n>" ]
}
```

donde <valor>, <id> y <nombre> son los valores de los atributos correspondientes.

4. Crear la clase que representa las características de un edificio.

Nombre de la clase: `Edificio`

Atributos:

- `id` (`String`): identificador único del edificio.
- `rango (int[])`: indica el número de la planta inferior y la superior del edificio, donde estos valores deberán estar comprendidos entre -2 y 6.
- `plantas (HashMap<Integer, Planta>)`: plantas que posee el edificio.
- `presupuesto` (`float`): presupuesto total del edificio. Su valor es la suma del presupuesto de todas las plantas del edificio.

Constructores:

Se debe implementar el siguiente constructor, en el que se deben cumplir las restricciones de valores indicadas para los setters de los atributos:

- Constructor con los siguientes argumentos: `id` (`String`), `rango (int[])` y `plantas` (`ArrayList<Planta>`), donde los valores del atributo `rango[]` deben cumplir las restricciones para el atributo; y el valor del atributo `numero` de las `plantas` debe estar comprendido entre el rango especificado. Por lo tanto, en caso de que alguna planta no tenga ese número, no formará parte de las plantas del edificio. Además, también hay que tener en cuenta que no puede haber una planta con `numero` repetido.
- Constructor con los siguientes argumentos: `id` (`String`).

Getters/Setters:

Todos los atributos de la clase deben tener los correspondientes getters implementados.

Se implementará el setter para el atributo `rango[]`, cumpliendo las restricciones indicadas en los constructores.

Métodos funcionales:

- `public boolean darAltaPlanta(Planta planta)`: dar de alta una `planta`, teniendo en cuenta las restricciones indicadas en los constructores.
- `public int[] plantasNoAlta()`: devuelve los número de las plantas que aún no han sido dadas de alta en el edificio.
- `public Set<Planta> plantasMasCaras ()`: obtiene las plantas que tienen un presupuesto mayor.
- `public HashMap<Integer, Set<Estancia>> plantasMasProblemas()`: devuelve el conjunto de estancias que tiene un mayor número de problemas en cada planta, donde la clave será el número de planta y los valores serán el conjunto de estancias de cada planta que tienen un mayor número de problemas (en el caso de que haya más de una estancia con un mismo número).
- `public String toString()`: la representación en cadena de texto que debe seguirá la siguiente forma:

```
{
 "id": "<id>",
 "plantas": [ { "<numero_1>", "<numero_2>,..., "<numero_n> } ],
 "presupuesto" : "<valor>"
}
```

donde <valor>, <id> y <numero> son los valores de los atributos correspondientes.

5. Crear la clase principal del programa en la que se realizarán las siguientes operaciones:

Nombre de la clase: `PrincipalB2E3`

Operaciones a realizar en la función `main()`:

- **a.** Crear los sensores indicados en el fichero Excel B2E3_instancias.xls.
- **b.** Crear las plantas indicadas en el fichero Excel B2E3_instancias.xls usando el constructor que tiene como argumentos el `numero` y el `tipo`.
- **c.** Mostrar por pantalla las características de las plantas creadas en (**b**).
- **d.** Crear las estancias indicadas en el fichero Excel B2E3_instancias.xls, dando de alta los sensores que van del `sensor1` al `sensor17` usando el método que permite dar de alta a un sensor a partir de su instancia. Los otros sensores se darán de alta en subsiguientes apartados.
- **e.** Mostrar por pantalla las características de las estancias creadas en (**d**).
- **f.** En las plantas `planta_1`, `planta0` y `planta1`, dar de alta las estancias que van de `estancia1` a `estancia10` usando para ello el método que acepta como argumentos un `ArrayList<Estancia>`.
- **g.** Mostrar por pantalla las características de las plantas creadas en (**b**).
- **h.** En la planta `planta1`, dar de alta los sensores `sensor18` y `sensor23` usando el método que tiene como argumentos el `id` de la estancia y `ArrayList<Sensor>`.
- **i.** En la planta `planta1`, dar de alta el sensor `sensor24` usando el método que tiene como argumentos el `id` de la estancia y `Sensor`.
- **j.** Mostrar por pantalla las características de la `planta1`.
- **k.** En la planta `planta_1`, dar de alta los sensores `sensor19`, `sensor20` y `sensor21` usando el método que tiene como argumentos la instancia de `Sensor`.
- **l.** Mostrar por pantalla las características de la `planta_1`.
- **m.** En la planta `planta2`, dar de alta las estancias `estancia11`, `estancia12` y `estancia13` usando el método que tiene como argumento una instancia de `Estancia`.
- **n.** En la planta `planta2`, dar de alta el sensor `sensor25` usando el método que tiene como argumentos la instancia de `Sensor`.
- **o.** Mostrar por pantalla las características de la `planta2`.
- **p.** En la planta `planta0`, dar de alta el sensor `sensor22` usando el método que tiene como argumentos la instancia de `Sensor`.
- **q.** Mostrar por pantalla las características de la `planta0`.
- **r.** Crear una instancia de `Edificio` a partir del constructor con argumentos de identificador `edificio1`, un rango de [-1, 2] y las plantas `planta_1`, `planta1` y `planta2`.
- **s.** Mostrar por pantalla las plantas que aún no están asignadas en el edificio.
- **t.** Mostrar por pantalla las características de la `planta2`.
- **u.** Dar de baja el `sensor8` de la `planta2` y sustituirlo por `sensor26`.
- **v.** Mostrar por pantalla las características de la `planta2`.
- **w.** Dar de baja el `sensor6` de la `planta1` y substituirlo por `sensor27`.
- **x.** Mostrar por pantalla las características de la `planta1`.

y. Dar de baja el `sensor10` de la `planta_1` y substituirlo por `sensor28`.

z. Mostrar por pantalla las características de la `planta_1`.

aa. Dar de baja el `sensor22` de la `planta0` y substituirlo por `sensor29`.

bb. Dar de baja el `sensor29` de la `planta0` y substituirlo por `sensor30`.

cc. Dar de baja el `sensor30` de la `planta0` y substituirlo por `sensor31`.

dd. Mostrar por pantalla las características de la `planta0`.

ee. Mostrar por pantalla los tipos de sensores más defectuosos de la `planta0`.

ff. Mostrar por pantalla los sensores de "humedad" que hay en la `planta2`.

gg. Mostrar por pantalla los sensores de "sonido" de la `planta1`.

hh. Dar de alta la `planta0` en el `edificio1`.

ii. Mostrar por pantalla las plantas que aún no están asignadas en el edificio.

jj. Mostrar las plantas del edificio con más problemas.

kk. Mostrar las plantas del edificio más caras.